

■ Architecture Decision Records

CliniDesk — Decisões Arquiteturais até a Sprint 2

Projeto: CliniDesk — NPI

Versão: Sprint 2 · 2026

Ano: 2026

Sobre este Documento

Este documento registra todas as **Architecture Decision Records (ADRs)** do projeto CliniDesk, adotando o formato padrão de mercado: *Contexto* → *Decisão* → *Justificativa* → *Consequências*. Cada ADR captura o raciocínio por trás de uma decisão técnica relevante, servindo como guia histórico para a equipe atual e futura.

■ Aceito — em uso na produção

■■ Substituído — ver ADR posterior

■ Rejeitado

Índice de Decisões

ADR	Título	Status	Data
001	Arquitetura de Monolito Modular	■ Aceito	30/04/2026
002	Banco de Dados NoSQL (MongoDB)	■ Aceito	30/04/2026
003	Autenticação JWT com Username	■■ Atualizado por ADR 005	30/04/2026
004	Variáveis de Ambiente (.env)	■ Aceito	30/04/2026
005	JWT via Cookies HttpOnly (segurança XSS)	■ Aceito	Sprint 2

Decisões Arquiteturais

ADR
001

Arquitetura de Monolito Modular

■ 30/04/2026

Status: ■ Aceito

Contexto

Precisávamos decidir a estrutura organizacional do sistema CliniDesk para o MVP do NPI. As opções consideradas foram Microserviços ou Monolito Modular.

Decisão

Optamos por uma Arquitetura de Monolito Modular, onde o Backend (Node/Express) gerencia todas as regras de negócio em um único serviço, e o Frontend (React) consome essa API via HTTP.

Justificativa

- 1. Redução de Complexidade:** A comunicação entre múltiplos serviços (microserviços) aumentaria drasticamente o tempo de entrega e a curva de aprendizado da equipe.
- 2. Custo de Infraestrutura:** Um monolito é mais barato e simples de hospedar para um MVP de clínica local.
- 3. Consistência de Dados:** Facilita transações atômicas e garante sincronização entre dados de pacientes e usuários.

Consequências

■ Positivo

- Módulos críticos como Agendamento e Prontuário podem ser escalados de forma independente caso o CliniDesk sofra picos de acesso em horários comerciais.
- Menor tempo de entrega do MVP com arquitetura mais simples e conhecida pela equipe.

■■ Negativo / Trade-off

- Escala total do sistema exige migração futura para microserviços se o volume crescer significativamente.

ADR
002

Banco de Dados NoSQL — MongoDB

■ 30/04/2026

Status: ■ Aceito

Contexto

O sistema precisa armazenar agendamentos que podem sofrer alterações frequentes de estrutura (novos campos de observação, tipos de convênio, etc.). O projeto é um MVP com requisitos em evolução.

Decisão

Utilizar o **MongoDB** como banco de dados principal, integrado via Mongoose e hospedado no MongoDB Atlas.

Justificativa

- 1. Esquema Flexível (Schema-less):** Como o projeto é um MVP, os requisitos mudam rapidamente. O MongoDB permite adicionar campos sem migrations complexas como em SQL.
- 2. Produtividade da Equipe:** O formato de documentos (BSON) é similar ao JSON do JavaScript, diminuindo a curva de aprendizado.
- 3. Integração Nativa:** A stack Node + Mongoose + MongoDB é extremamente estável e bem documentada.

Consequências

■ Positivo

- Conformidade ACID: garante que atualizações em prontuários sejam processadas de forma segura.
- Facilita a geração de relatórios gerenciais com queries de agregação nativas do MongoDB.

■■ Negativo / Trade-off

- Consultas relacionais complexas (JOINS) exigem uso de \$lookup, menos intuitivo que SQL.

ADR
003

Autenticação JWT com Login por
Username

■ 30/04/2026

Status: ■■ Atualizado — ver ADR 005

Contexto

Necessidade de identificar usuários na recepção de forma rápida, sem exigir e-mails longos, e manter a sessão segura.

Decisão

Implementar autenticação Stateless via JWT, substituindo o campo e-mail por username (iniciais/login curto, ex: *erlon* em vez de *erlon.silva@clinica.com.br*). O token era inicialmente armazenado no frontend via localStorage/header Authorization.

Justificativa

- 1. Agilidade Operacional:** A recepcionista precisa de velocidade — username curto é mais rápido.
- 2. Escalabilidade (Stateless):** O servidor não precisa guardar estados de sessão na memória.

3. Segurança: O JWT permite expiração automática e assinatura digital.

Consequências

■ Positivo

- Reduz consultas ao banco apenas para verificar se o usuário está logado.
- Username curto é mais ergonômico para ambientes de recepção.

■ Negativo / Trade-off

- Token armazenado em localStorage era acessível via JavaScript, expondo-o a ataques XSS. Problema resolvido na ADR 005.
- Exige implementação de lógica de Refresh Token para renovação de sessão.

ADR
004

Padronização via Variáveis de
Ambiente (.env)

■ 30/04/2026

Status: ■ Aceito

Contexto

O projeto enfrentava erros de conexão (undefined) e riscos de segurança ao expor senhas do banco de dados no código-fonte comprometido no controle de versão.

Decisão

Adotar o uso obrigatório de arquivos **.env** e da biblioteca dotenv em todos os ambientes de desenvolvimento.

Justificativa

- 1. Segurança (DevSecOps):** Impede que credenciais críticas (MONGO_URI, JWT_SECRET) sejam enviadas ao GitHub.
- 2. Portabilidade:** Facilita a troca de ambiente (Desenvolvimento vs. Produção) sem alterar código lógico.
- 3. Padronização da Equipe:** Todos seguem o modelo .env.example, evitando o problema 'na minha máquina funciona'.

Consequências

■ Positivo

- Cria um padrão claro: onde cada configuração deve ficar, reduzindo confusão durante o desenvolvimento.
- Compatível nativamente com Docker via docker-compose env_file.

■ Negativo / Trade-off

- Curva de aprendizado inicial para membros menos experientes da equipe.

ADR
005

Migração do JWT para Cookies HttpOnly

■ Sprint 2

Status: ■ Aceito

Contexto

No modelo anterior (ADR 003), o frontend era responsável pelo ciclo de vida completo do token JWT (armazenamento, gerenciamento e envio). Isso expunha o token ao escopo do JavaScript, criando vulnerabilidades XSS onde um script malicioso poderia roubar o token de autenticação.

Decisão

Transferir a responsabilidade de armazenamento e transporte do token para o navegador, utilizando **cookies seguros** com as flags HttpOnly, Secure e SameSite. O frontend torna-se agnóstico em relação ao token.

Justificativa

1. **Mitigação de XSS:** Token inacessível via JavaScript — scripts maliciosos não conseguem lê-lo.
2. **Simplificação do Frontend:** Elimina lógica complexa de gerenciamento de estado de autenticação.
3. **Desacoplamento:** Frontend foca na interface; navegador e backend resolvem o transporte da identidade.

Consequências

■ Positivo

- Redução da superfície de ataque — token não aparece no DevTools > Application > Local Storage.
- Sessões multi-abas nativas: compartilhamento automático de sessão entre abas do navegador.
- Fluxo reativo: frontend dispara chamada → backend valida → interceptor global captura 401 e redireciona ao login.

■■ Negativo / Trade-off

- Requer configuração correta de CORS e SameSite para não bloquear requisições cross-origin em produção.